```
fseek ( fp, -recsize, SEEK_CUR ) ;
```

Here, **-recsize** moves the pointer back by **recsize** bytes from the current position. **SEEK_CUR** is a macro defined in "stdio.h".

Similarly, the following **fseek( )** would place the pointer beyond the last record in the file.

```
fseek ( fp, 0, SEEK_END ) ;
```

In fact **-recsize** or **0** are just the offsets that tell the compiler by how many bytes should the pointer be moved from a particular position. The third argument could be **SEEK_END**, **SEEK_CUR** or **SEEK_SET**. All these act as a reference from which the pointer should be offset. **SEEK_END** means move the pointer from the end of the file, **SEEK_CUR** means move the pointer with reference to its current position and **SEEK_SET** means move the pointer with reference to the beginning of the file.

If we wish to know where the pointer is positioned right now, we can use the function **ftell( )**. It returns this position as a **long int** which is an offset from the beginning of the file. The value returned by **ftell( )** can be used in subsequent calls to **fseek( )**. A sample call to **ftell( )** is shown below:

```
position = ftell ( fp ) ;
```

where **position** is a **long int**.

Low Level Disk I/O

In low level disk I/O, data cannot be written as individual characters, or as strings or as formatted data. There is only one way data can be written or read in low level disk I/O functions—as a buffer full of bytes.

Writing a buffer full of data resembles the **fwrite( )** function. However, unlike **fwrite( )**, the programmer must set up the buffer for the data, place the appropriate values in it before writing, and take them out after writing. Thus, the buffer in the low level I/O functions is very much a part of the program, rather than being invisible as in high level disk I/O functions.

Low level disk I/O functions offer following advantages:

(a) Since these functions parallel the methods that the OS uses to write to the disk, they are more efficient than the high level disk I/O functions.

(b) Since there are fewer layers of routines to go through, low level I/O functions operate faster than their high level counterparts.

Let us now write a program that uses low level disk input/output functions.

A Low Level File-copy Program

Earlier we had written a program to copy the contents of one file to another. In that program we had read the file character by character using **fgetc( )**. Each character that was read was written into the target file using **fputc( )**. Instead of performing the I/O on a character by character basis we can read a chunk of bytes from the source file and then write this chunk into the target file. While doing so the chunk would be read into the buffer and would be written to the file from the buffer. While doing so we would manage the buffer ourselves, rather than relying on the library functions to do so. This is what is low-level about this program. Here is a program which shows how this can be done.

```
/* File-copy program which copies text, .com and .exe files */
#include "fcntl.h"
#include "types.h"  /* if present in sys directory use
```

```
                        "c:tc\\include\\sys\\types.h" */
#include "stat.h"  /* if present in sys directory use
                        "c:\\tc\\include\\sys\\stat.h" */

main ( int  argc, char  *argv[ ] )
{
    char  buffer[ 512 ], source [ 128 ], target [ 128 ] ;
    int  inhandle, outhandle, bytes ;

    printf ( "\nEnter source file name" ) ;
    gets ( source ) ;

    inhandle = open ( source, O_RDONLY | O_BINARY ) ;
    if ( inhandle == -1 )
    {
        puts ( "Cannot open file" ) ;
        exit( ) ;
    }

    printf ( "\nEnter target file name" ) ;
    gets ( target ) ;

    outhandle = open ( target, O_CREAT | O_BINARY | O_WRONLY,
                        S_IWRITE ) ;
    if ( inhandle == -1 )
    {
        puts ( "Cannot open file" ) ;
        close ( inhandle ) ;
        exit( ) ;
    }

    while ( 1 )
    {
        bytes = read ( inhandle, buffer, 512 ) ;

        if ( bytes > 0 )
            write ( outhandle, buffer, bytes ) ;
        else
```

```
            break ;
    }

    close ( inhandle ) ;
    close ( outhandle ) ;
}
```

Declaring the Buffer

The first difference that you will notice in this program is that we declare a character buffer,

```
char  buffer[512] ;
```

This is the buffer in which the data read from the disk will be placed. The size of this buffer is important for efficient operation. Depending on the operating system, buffers of certain sizes are handled more efficiently than others.

Opening a File

We have opened two files in our program, one is the source file from which we read the information, and the other is the target file into which we write the information read from the source file.

As in high level disk I/O, the file must be opened before we can access it. This is done using the statement,

```
inhandle = open ( source, O_RDONLY | O_BINARY ) ;
```

We open the file for the same reason as we did earlier—to establish communication with operating system about the file. As usual, we have to supply to **open( )**, the filename and the mode in which we want to open the file. The possible file opening modes are given below:

O_APPEND - Opens a file for appending

O_CREAT	-	Creates a new file for writing (has no effect if file already exists)
O_RDONLY	-	Creates a new file for reading only
O_RDWR	-	Creates a file for both reading and writing
O_WRONLY	-	Creates a file for writing only
O_BINARY	-	Creates a file in binary mode
O_TEXT	-	Creates a file in text mode

These 'O-flags' are defined in the file "fcntl.h". So this file must be included in the program while usng low level disk I/O. Note that the file "stdio.h" is not necessary for low level disk I/O. When two or more O-flags are used together, they are combined using the bitwise OR operator (|). Chapter 14 discusses bitwise operators in detail.

The other statement used in our program to open the file is,

```
outhandle = open ( target, O_CREAT | O_BINARY | O_WRONLY,
                   S_IWRITE ) ;
```

Note that since the target file is not existing when it is being opened we have used the O_CREAT flag, and since we want to write to the file and not read from it, therefore we have used O_WRONLY. And finally, since we want to open the file in binary mode we have used O_BINARY.

Whenever O_CREAT flag is used, another argument must be added to **open( )** function to indicate the read/write status of the file to be created. This argument is called 'permission argument'. Permission arguments could be any of the following:

S_IWRITE	-	Writing to the file permitted
S_IREAD	-	Reading from the file permitted

To use these permissions, both the files "types.h" and "stat.h" must be **#include**d in the program alongwith "fcntl.h".

File Handles

Instead of returning a FILE pointer as **fopen( )** did, in low level disk I/O, **open( )** returns an integer value called 'file handle'. This is a number assigned to a particular file, which is used thereafter to refer to the file. If **open( )** returns a value of -1, it means that the file couldn't be successfully opened.

Interaction between Buffer and File

The following statement reads the file or as much of it as will fit into the buffer:

```
bytes = read ( inhandle, buffer, 512 ) ;
```

The **read( )** function takes three arguments. The first argument is the file handle, the second is the address of the buffer and the third is the maximum number of bytes we want to read.

The **read( )** function returns the number of bytes actually read. This is an important number, since it may very well be less than the buffer size (512 bytes), and we will need to know just how full the buffer is before we can do anything with its contents. In our program we have assigned this number to the variable **bytes**.

For copying the file, we must use both the **read( )** and the **write( )** functions in a **while** loop. The **read( )** function returns the number of bytes actually read. This is assigned to the variable **bytes**. This value will be equal to the buffer size (512 bytes) until the end of file, when the buffer will only be partially full. The variable **bytes** therefore is used to tell **write( )**, as to how many bytes to write from the buffer to the target file.

Note that when large buffers are used they must be made global variables otherwise stack overflow occurs.

I/O Under Windows

As said earlier I/O in C is carried out using functions present in the library that comes with the C compiler targeted for a specific OS. Windows permits several applications to use the same screen simultaneously. Hence there is a possibility that what is written by one application to the console may get overwritten by the output sent by another application to the console. To avoid such situations Windows has completely abandoned console I/O functions. It uses a separate mechanism to send output to a window representing an application. The details of this mechanism are discussed in Chapter 17.

Though under Windows console I/O functions are not used, still functions like **fprintf( )**, **fscanf( )**, **fread( )**, **fwrite( )**, **sprintf( )**, **sscanf( )** work exactly same under Windows as well.

Summary

(a) File I/O can be performed on a character by character basis, a line by line basis, a record by record basis or a chunk by chunk basis.
(b) Different operations that can be performed on a file are—creation of a new file, opening an existing file, reading from a file, writing to a file, moving to a specific location in a file (seeking) and closing a file.
(c) File I/O is done using a buffer to improve the efficiency.
(d) A file can be a text file or a binary file depending upon its contents.
(e) Library functions convert **\n** to **\r\n** or vice versa while writing/reading to/from a file.

(f) Many library functions convert a number to a numeric string before writing it to a file, thereby using more space on disk. This can be avoided using functions **fread( )** and **fwrite( )**.

(g) In low level file I/O we can do the buffer management ourselves.

Exercise

[A] Point out the errors, if any, in the following programs:

(a)
```
#include "stdio.h"
main( )
{
    FILE *fp ;
    openfile ( "Myfile.txt", fp ) ;
    if  ( fp == NULL )
        printf ( "Unable to open file…" ) ;
}

openfile ( char *fn, FILE **f )
{
    *f = fopen ( fn, "r" ) ;
}
```

(b)
```
#include "stdio.h"
main( )
{
    FILE  *fp ;
    char  c ;
    fp = fopen ( "TRY.C" ,"r") ;
    if ( fp == null )
    {
        puts ( "Cannot open file" ) ;
        exit( ) ;
    }
    while ( ( c = getc ( fp ) ) != EOF )
        putch ( c ) ;
    fclose ( fp ) ;
```

```
    }
(c) main( )
    {
        char fname[ ] = "c:\\students.dat" ;
        FILE *fp ;
        fp = fopen ( fname, "tr" ) ;
        if ( fp == NULL )
            printf ( "\nUnable to open file..." ) ;
    }
(d) main( )
    {
        FILE  *fp ;
        char  str[80] ;
        fp = fopen ( "TRY.C", "r" ) ;
        while ( fgets ( str, 80, fp ) != EOF )
            fputs ( str ) ;
        fclose ( fp ) ;
    }
(e) #include "stdio.h"
    {
        unsigned char ;
        FILE *fp ;

        fp = fopen ( "trial", "r" ) ;
        while ( ( ch = getc ( fp ) ) != EOF )
            printf ( "%c", ch ) ;

        fclose ( fp ) ;
    }
(f) main( )
    {
        FILE  *fp ;
        char  name[25] ;
        int  age ;

        fp = fopen ( "YOURS", "r" ) ;
```

```
        while ( fscanf ( fp, "%s %d", name, &age ) != NULL )
        fclose ( fp ) ;
    }
```

(g)
```
    main( )
    {
        FILE  *fp ;
        char  names[20] ;
        int  i ;
        fp = fopen ( "students.c", "wb" ) ;
        for ( i = 0 ; i <= 10 ; i++ )
        {
            puts ( "\nEnter name " ) ;
            gets ( name ) ;
            fwrite ( name, size of ( name ), 1, fp ) ;
        }
        close ( fp ) ;
    }
```

(h)
```
    main( )
    {
        FILE  *fp ;
        char  name[20] = "Ajay" ;
        int  i ;
        fp = fopen ( "students.c", "r" ) ;
        for ( i = 0 ; i <= 10 ; i++ )
            fwrite ( name, sizeof ( name ), 1, fp ) ;
        close ( fp ) ;
    }
```

(i)
```
    #include "fcntl.h"
    main( )
    {
        int  fp ;
        fp = open ( "pr22.c" , "r" ) ;
        if ( fp == -1 )
            puts ( "cannot open file" ) ;
        else
            close ( fp ) ;
```

```
}
```

(j)
```
main( )
{
    int  fp ;
    fp = fopen ( "students.c", READ | BINARY ) ;
    if ( fp == -1 )
        puts ( "cannot open file" ) ;
    else
        close ( fp ) ;
}
```

[B] Answer the following:

(a) The macro FILE is defined in which of the following files:

1. stdlib.h
2. stdio.c
3. io.h
4. stdio.h

(b) If a file contains the line "I am a boy\r\n" then on reading this line into the array **str[ ]** using **fgets( )** what would **str[ ]** contain?

1. I am a boy\r\n\0
2. I am a boy\r\0
3. I am a boy\n\0
4. I am a boy

(c) State True or False:

1. The disadvantage of High Level Disk I/O functions is that the programmer has to manage the buffers.
2. If a file is opened for reading it is necessary that the file must exist.
3. If a file opened for writing already exists its contents would be overwritten.

4. For opening a file in append mode it is necessary that the file should exist.

(d) On opening a file for reading which of the following activities are performed:

1. The disk is searched for existence of the file.
2. The file is brought into memory.
3. A pointer is set up which points to the first character in the file.
4. All the above.

(e) Is it necessary that a file created in text mode must always be opened in text mode for subsequent operations?

(f) State True or False:

A file opened in binary mode and read using **fgetc( )** would report the same number of characters in the file as reported by DOS's DIR command.

(g) While using the statement,

```
fp = fopen ( "myfile.c", "r" ) ;
```

what happens if,

- 'myfile.c' does not exist on the disk
- 'myfile.c' exists on the disk

(h) What is the purpose of the library function **fflush( )**?

(i) While using the statement,

```
fp = fopen ( "myfile.c", "wb" ) ;
```

what happens if,

- 'myfile.c' does not exist on the disk.
- 'myfile.c' exists on the disk

(j) A floating-point array contains percentage marks obtained by students in an examination. To store these marks in a file 'marks.c', in which mode would you open the file and why?

[C] Attempt the following:

(a) Write a program to read a file and display contents with its line numbers.

(b) Write a program to find the size of a text file without traversing it character by character.

(c) Write a program to add the contents of one file at the end of another.

(d) Suppose a file contains student's records with each record containing name and age of a student. Write a program to read these records and display them in sorted order by name.

(e) Write a program to copy one file to another. While doing so replace all lowercase characters to their equivalent uppercase characters.

(f) Write a program that merges lines alternately from two files and writes the results to new file. If one file has less number of lines than the other, the remaining lines from the larger file should be simply copied into the target file.

(g) Write a program to display the contents of a text file on the screen. Make following provisions:

Display the contents inside a box drawn with opposite corner co-ordinates being (0, 1) and (79, 23). Display the name of the file whose contents are being displayed, and the page numbers in the zeroth row. The moment one screenful of file has been displayed, flash a message 'Press any key...' in 24^{th} row. When a key is hit, the next page's contents should be displayed, and so on till the end of file.

(h) Write a program to encrypt/decrypt a file using:

(1) An offset cipher: In an offset cipher each character from the source file is offset with a fixed value and then written to the target file.

For example, if character read from the source file is 'A', then convert this into a new character by offsetting 'A' by a fixed value, say 128, and then writing the new character to the target file.

(2) A substitution cipher: In this each character read from the source file is substituted by a corresponding predetermined character and this character is written to the target file.

For example, if character 'A' is read from the source file, and if we have decided that every 'A' is to be substituted by '!', then a '!' would be written to the target file in place of every 'A' Similarly, every 'B' would be substituted by '5' and so on.

(i) In the file 'CUSTOMER.DAT' there are 100 records with the following structure:

```
struct customer
{
    int  accno ;
    char name[30] ;
    float  balance ;
} ;
```

In another file 'TRANSACTIONS.DAT' there are several records with the following structure:

```
struct trans
{
    int  accno ,
    char trans_type ;
```

```
    float amount ;
} ;
```

The parameter **trans_type** contains D/W indicating deposit or withdrawal of amount. Write a program to update 'CUSTOMER.DAT' file, i.e. if the **trans_type** is 'D' then update the **balance** of 'CUSTOMER.DAT' by adding **amount** to balance for the corresponding **accno**. Similarly, if **trans_type** is 'W' then subtract the **amount** from **balance**. However, while subtracting the amount make sure that the amount should not get overdrawn, i.e. at least 100 Rs. Should remain in the account.

(j) There are 100 records present in a file with the following structure:

```
struct date
{
    int d, m, y ;
} ;

struct employee
{
    int  empcode[6] ;
    char empname[20] ;
    struct date join_date ;
    float salary ;
} ;
```

Write a program to read these records, arrange them in ascending order of **join_date** and write them in to a target file.

(k) A hospital keeps a file of blood donors in which each record has the format:
Name: 20 Columns
Address: 40 Columns

Age: 2 Columns
Blood Type: 1 Column (Type 1, 2, 3 or 4)

Write a program to read the file and print a list of all blood donors whose age is below 25 and blood is type 2.

(l) Given a list of names of students in a class, write a program to store the names in a file on disk. Make a provision to display the $\mathbf{n}^{th}$ name in the list (**n** is data to be read) and to display all names starting with S.

(m) Assume that a Master file contains two fields, Roll no. and name of the student. At the end of the year, a set of students join the class and another set leaves. A Transaction file contains the roll numbers and an appropriate code to add or delete a student.

Write a program to create another file that contains the updated list of names and roll numbers. Assume that the Master file and the Transaction file are arranged in ascending order by roll numbers. The updated file should also be in ascending order by roll numbers.

(n) In a small firm employee numbers are given in serial numerical order, that is 1, 2, 3, etc.

- Create a file of employee data with following information: employee number, name, sex, gross salary.
- If more employees join, append their data to the file.
- If an employee with serial number 25 (say) leaves, delete the record by making gross salary 0.
- If some employee's gross salary increases, retrieve the record and update the salary.

Write a program to implement the above operations.

(o) Given a text file, write a program to create another text file deleting the words "a", "the", "an" and replacing each one of them with a blank space.

(p) You are given a data file EMPLOYEE.DAT with the following record structure:

```
struct employee {
                    int  empno ;
                    char  name[30] ;
                    int  basic, grade ;
                } ;
```

Every employee has a unique **empno** and there are supposed to be no gaps between employee numbers. Records are entered into the data file in ascending order of employee number, **empno**. It is intended to check whether there are missing employee numbers. Write a program segment to read the data file records sequentially and display the list of missing employee numbers.

(q) Write a program to carry out the following:

- To read a text file "TRIAL.TXT" consisting of a maximum of 50 lines of text, each line with a maximum of 80 characters.
- Count and display the number of words contained in the file.
- Display the total number of four letter words in the text file.

Assume that the end of a word may be a space, comma or a full-stop followed by one or more spaces or a newline character.

(r) Write a program to read a list of words, sort the words in alphabetical order and display them one word per line. Also give the total number of words in the list. Output format should be:
Total Number of words in the list is _____
Alphabetical listing of words is:

Assume the end of the list is indicated by **ZZZZZZ** and there are maximum in 25 words in the Text file.

(s) Write a program to carry out the following:

(a) Read a text file 'INPUT.TXT'
(b) Print each word in reverse order

Example,

Input: INDIA IS MY COUNTRY
Output: AIDNI SI YM YRTNUOC

Assume that each word length is maximum of 10 characters and each word is separated by newline/blank characters.

(t) Write a C program to read a large text file 'NOTES.TXT' and print it on the printer in cut-sheets, introducing page breaks at the end of every 50 lines and a pause message on the screen at the end of every page for the user to change the paper.

13 More Issues In Input/Output

- Using *argc* and *argv*
- Detecting Errors in Reading/Writing
- Standard I/O Devices
- I/O Redirection
  - Redirecting the Output
  - Redirecting the Input
  - Both Ways at Once
- Summary
- Exercise

In Chapters 11 and 12 we saw how Console I/O and File I/O are done in C. There are still some more issues related with input/output that remain to be understood. These issues help in making the I/O operations more elegant.

Using *argc* and *argv*

To execute the file-copy programs that we saw in Chapter 12 we are required to first type the program, compile it, and then execute it. This program can be improved in two ways:

(a) There should be no need to compile the program every time to use the file-copy utility. It means the program must be executable at command prompt (A> or C> if you are using MS-DOS, Start | Run dialog if you are using Windows and $ prompt if you are using Unix).

(b) Instead of the program prompting us to enter the source and target filenames, we must be able to supply them at command prompt, in the form:

```
filecopy PR1.C PR2.C
```

where, PR1.C is the source filename and PR2.C is the target filename.

The first improvement is simple. In MS-DOS, the executable file (the one which can be executed at command prompt and has an extension .EXE) can be created in Turbo C/C++ by using the key F9 to compile the program. In VC++ compiler under Windows same can be done by using F7 to compile the program. Under Unix this is not required since in Unix every time we compile a program we always get an executable file.

The second improvement is possible by passing the source filename and target filename to the function **main( )**. This is illustrated below:

```
#include "stdio.h"
main ( int  argc, char  *argv[ ] )
{
      FILE  *fs, *ft ;
      char  ch ;

      if ( argc != 3 )
      {
            puts ( "Improper number of arguments" ) ;
            exit( ) ;
      }

      fs = fopen ( argv[1], "r" ) ;
      if ( fs == NULL )
      {
            puts ( "Cannot open source file" ) ;
            exit( ) ;
      }

      ft = fopen ( argv[2], "w" ) ;
      if ( ft == NULL )
      {
            puts ( "Cannot open target file" ) ;
            fclose ( fs ) ;
            exit( ) ;
      }

      while ( 1 )
      {
            ch = fgetc ( fs ) ;

            if ( ch == EOF )
                  break ;
            else
                  fputc ( ch, ft ) ;
      }
```

```
    fclose ( fs ) ;
    fclose ( ft ) ;
}
```

The arguments that we pass on to **main( )** at the command prompt are called command line arguments. The function **main( )** can have two arguments, traditionally named as **argc** and **argv**. Out of these, **argv** is an array of pointers to strings and **argc** is an **int** whose value is equal to the number of strings to which **argv** points. When the program is executed, the strings on the command line are passed to **main( )**. More precisely, the strings at the command line are stored in memory and address of the first string is stored in **argv[0]**, address of the second string is stored in **argv[1]** and so on. The argument **argc** is set to the number of strings given on the command line. For example, in our sample program, if at the command prompt we give,

```
filecopy  PR1.C  PR2.C
```

then,

argc would contain 3
argv[0] would contain base address of the string "filecopy"
argv[1] would contain base address of the string "PR1.C"
argv[2] would contain base address of the string "PR2.C"

Whenever we pass arguments to **main( )**, it is a good habit to check whether the correct number of arguments have been passed on to **main( )** or not. In our program this has been done through,

```
if ( argc != 3 )
{
    printf ( "Improper number of arguments" ) ;
    exit( ) ;
}
```

Rest of the program is same as the earlier file-copy program. This program is better than the earlier file-copy program on two counts:

(a) There is no need to recompile the program every time we want to use this utility. It can be executed at command prompt.
(b) We are able to pass source file name and target file name to **main( )**, and utilize them in **main( )**.

One final comment... the **while** loop that we have used in our program can be written in a more compact form, as shown below:

```
while ( ( ch = fgetc ( fs ) ) != EOF )
    fputc ( ch, ft ) ;
```

This avoids the usage of an indefinite loop and a **break** statement to come out of this loop. Here, first **fgetc (fs)** gets the character from the file, assigns it to the variable **ch**, and then **ch** is compared against **EOF**. Remember that it is necessary to put the expression

```
ch = fgetc ( fs )
```

within a pair of parentheses, so that first the character read is assigned to variable **ch** and then it is compared with **EOF**.

There is one more way of writing the **while** loop. It is shown below:

```
while ( !feof ( fs ) )
{
    ch = fgetc ( fs ) ;
    fputc ( ch, ft ) ;
}
```

Here, **feof( )** is a macro which returns a 0 if end of file is not reached. Hence we use the **!** operator to negate this 0 to the truth value. When the end of file is reached **feof( )** returns a non-zero

value, **!** makes it 0 and since now the condition evaluates to false the **while** loop gets terminated.

Note that in each one of them the following three methods for opening a file are same, since in each one of them, essentially a base address of the string (pointer to a string) is being passed to **fopen( )**.

```
fs = fopen ( "PR1.C" , "r" ) ;
fs = fopen ( filename, "r" ) ;
fs = fopen ( argv[1] , "r" ) ;
```

Detecting Errors in Reading/Writing

Not at all times when we perform a read or write operation on a file are we successful in doing so. Naturally there must be a provision to test whether our attempt to read/write was successful or not.

The standard library function **ferror( )** reports any error that might have occurred during a read/write operation on a file. It returns a zero if the read/write is successful and a non-zero value in case of a failure. The following program illustrates the usage of **ferror( )**.

```
#include "stdio.h"
main( )
{
    FILE  *fp ;
    char  ch ;

    fp = fopen ( "TRIAL", "w" ) ;

    while ( !feof ( fp ) )
    {
        ch = fgetc ( fp ) ;
        if ( ferror( ) )
        {
```

```
            printf ( "Error in reading file" ) ;
            break ;
        }
        else
            printf ( "%c", ch ) ;
    }

    fclose ( fp ) ;
}
```

In this program the **fgetc( )** function would obviously fail first time around since the file has been opened for writing, whereas **fgetc( )** is attempting to read from the file. The moment the error occurs **ferror( )** returns a non-zero value and the **if** block gets executed. Instead of printing the error message using **printf( )** we can use the standard library function **perror( )** which prints the error message specified by the compiler. Thus in the above program the **perror( )** function can be used as shown below.

```
if ( ferror( ) )
{
    perror ( "TRIAL" ) ;
    break ;
}
```

Note that when the error occurs the error message that is displayed is:

```
TRIAL: Permission denied
```

This means we can precede the system error message with any message of our choice. In our program we have just displayed the filename in place of the error message.

Standard I/O Devices

To perform reading or writing operations on a file we need to use the function **fopen( )**, which sets up a file pointer to refer to this file. Most OSs also predefine pointers for three standard files. To access these pointers we need not use **fopen( )**. These standard file pointers are shown in Figure 13.1

Standard File pointer	Description
stdin	standard input device (Keyboard)
stdout	standard output device (VDU)
stderr	standard error device (VDU)

Figure 13.1

Thus the statement **ch = fgetc (stdin)** would read a character from the keyboard rather than from a file. We can use this statement without any need to use **fopen( )** or **fclose( )** function calls.

Note that under MS-DOS two more standard file pointers are available—**stdprn** and **stdaux**. They stand for standard printing device and standard auxiliary device (serial port). The following program shows how to use the standard file pointers. It reads a file from the disk and prints it on the printer.

```
/* Prints file contents on printer */
#include "stdio.h"
main( )
{
    FILE  *fp ;
    char  ch ;
```

```
    fp = fopen ( "poem.txt", "r" ) ;

    if ( fp == NULL )
    {
        printf ( "Cannot open file" ) ;
        exit( ) ;
    }

    while ( ( ch = fgetc ( fp ) ) != EOF )
        fputc ( ch, stdprn ) ;

    fclose ( fp ) ;
}
```

The statement **fputc (ch, stdprn)** writes a character read from the file to the printer. Note that although we opened the file on the disk we didn't open **stdprn**, the printer. Standard files and their use in redirection have been dealt with in more details in the next section.

Note that these standard file pointers have been defined in the file "stdio.h". Therefore, it is necessary to include this file in the program that uses these standard file pointers.

I/O Redirection

Most operating systems incorporate a powerful feature that allows a program to read and write files, even when such a capability has not been incorporated in the program. This is done through a process called 'redirection'.

Normally a C program receives its input from the standard input device, which is assumed to be the keyboard, and sends its output to the standard output device, which is assumed to be the VDU. In other words, the OS makes certain assumptions about where input

should come from and where output should go. Redirection permits us to change these assumptions.

For example, using redirection the output of the program that normally goes to the VDU can be sent to the disk or the printer without really making a provision for it in the program. This is often a more convenient and flexible approach than providing a separate function in the program to write to the disk or printer. Similarly, redirection can be used to read information from disk file directly into a program, instead of receiving the input from keyboard.

To use redirection facility is to execute the program from the command prompt, inserting the redirection symbols at appropriate places. Let us understand this process with the help of a program.

Redirecting the Output

Let's see how we can redirect the output of a program, from the screen to a file. We'll start by considering the simple program shown below:

```
/* File name: util.c */
#include "stdio.h"<+>
main( )
{
      char  ch ;
      while ( ( ch = getc ( stdin ) ) != EOF )
            putc ( ch, stdout ) ;
}
```

On compiling this program we would get an executable file UTIL.EXE. Normally, when we execute this file, the **putc( )** function will cause whatever we type to be printed on screen, until we don't type Ctrl-Z, at which point the program will terminate, as

shown in the following sample run. The Ctrl-Z character is often called end of file character.

```
C>UTIL.EXE
perhaps I had a wicked childhood,
perhaps I had a miserable youth,
but somewhere in my wicked miserable past,
there must have been a moment of truth ^Z
C>
```

Now let's see what happens when we invoke this program from in a different way, using redirection:

```
C>UTIL.EXE > POEM.TXT
C>
```

Here we are causing the output to be redirected to the file POEM.TXT. Can we prove that this the output has indeed gone to the file POEM.TXT? Yes, by using the TYPE command as follows:

```
C>TYPE POEM.TXT
perhaps I had a wicked childhood,
perhaps I had a miserable youth,
but somewhere in my wicked miserable past,
there must have been a moment of truth
C>
```

There's the result of our typing sitting in the file. The redirection operator, '>', causes any output intended for the screen to be written to the file whose name follows the operator.

Note that the data to be redirected to a file doesn't need to be typed by a user at the keyboard; the program itself can generate it. Any output normally sent to the screen can be redirected to a disk file. As an example consider the following program for generating the ASCII table on screen:

```
/* File name: ascii.c*/
main( )
{
      int  ch ;

      for ( ch = 0 ; ch <= 255 ; ch++ )
            printf ( "\n%d %c", ch, ch ) ;
}
```

When this program is compiled and then executed at command prompt using the redirection operator,

```
C>ASCII.EXE > TABLE.TXT
```

the output is written to the file. This can be a useful capability any time you want to capture the output in a file, rather than displaying it on the screen.

DOS predefines a number of filenames for its own use. One of these names in PRN, which stands for the printer. Output can be redirected to the printer by using this filename. For example, if you invoke the “ascii.exe” program this way:

```
C>ASCII.EXE > PRN
```

the ASCII table will be printed on the printer.

Redirecting the Input

We can also redirect input to a program so that, instead of reading a character from the keyboard, a program reads it from a file. Let us now see how this can be done.

To redirect the input, we need to have a file containing something to be displayed. Suppose we use a file called NEWPOEM.TXT containing the following lines:

```
Let's start at the very beginning,
A very good place to start!
```

We'll assume that using some text editor these lines have been placed in the file NEWPOEM.TXT. Now, we use the input redirection operator '<' before the file, as shown below:

```
C>UTIL.EXE < NEWPOEM.TXT
Let's start at the very beginning,
A very good place to start!
C>
```

The lines are printed on the screen with no further effort on our part. Using redirection we've made our program UTIL.C perform the work of the TYPE command.

Both Ways at Once

Redirection of input and output can be used together; the input for a program can come from a file via redirection, at the same time its output can be redirected to a file. Such a program is called a filter. The following command demonstrates this process.

```
C>UTIL < NEWPOEM.TXT > POETRY.TXT
```

In this case our program receives the redirected input from the file NEWPOEM.TXT and instead of sending the output to the screen it would redirect it to the file POETRY.TXT.

Similarly to send the contents of the file NEWPOEM.TXT to the printer we can use the following command:

```
C>UTIL < NEWPOEM.TXT > PRN
```

While using such multiple redirections don't try to send output to the same file from which you are receiving input. This is because

the output file is erased before it's written to. So by the time we manage to receive the input from a file it is already erased.

Redirection can be a powerful tool for developing utility programs to examine or alter data in files. Thus, redirection is used to establish a relationship between a program and a file. Another OS operator can be used to relate two programs directly, so that the output of one is fed directly into another, with no files involved. This is called 'piping', and is done using the operator '|', called pipe. We won't pursue this topic, but you can read about it in the OS help/manual.

Summary

(a) We can pass parameters to a program at command line using the concept of 'command line arguments'.
(b) The command line argument **argv** contains values passed to the program, whereas, **argc** contains number of arguments.
(c) We can use the standard file pointer **stdin** to take input from standard input device such as keyboard.
(d) We can use the standard file pointer **stdout** to send output to the standard output device such as a monitor.
(e) We can use the standard file pointers **stdprn** and **stdaux** to interact with printer and auxiliary devices respectively.
(f) Redirection allows a program to read from or write to files at command prompt.
(g) The operators < and > are called redirection operators.

Exercise

[A] Answer the following:

(a) How will you use the following program to

- Copy the contents of one file into another.
- Print a file on the printer.
- Create a new file and add some text to it.

– Display the contents of an existing file.

```
#include "stdio.h"
main( )
{
    char ch, str[10] ;
    while ( ( ch = getc ( stdin ) ) != -1 )
    putc ( ch, stdout ) ;
}
```

(b) State True or False:

1. We can send arguments at command line even if we define **main( )** function without parameters.
2. To use standard file pointers we don't need to open the file using **fopen( )**.
3. Using **stdaux** we can send output to the printer if printer is attached to the serial port.
4. The zeroth element of the **argv** array is always the name of the exe file.

(c) Point out the errors, if any, in the following program

```
main ( int  ac, char  ( * ) av[ ] )
{
    printf ( "\n%d", ac ) ;
    printf ( "\n%s", av[0] ) ;
}
```

[B] Attempt the following:

(a) Write a program to carry out the following:

(a) Read a text file provided at command prompt
(b) Print each word in reverse order

For example if the file contains

INDIA IS MY COUNTRY

Output should be

AIDNI SI YM YRTNUOC

(b) Write a program using command line arguments to search for a word in a file and replace it with the specified word. The usage of the program is shown below.

C> change <old word> <new word> <filename>

(c) Write a program that can be used at command prompt as a calculating utility. The usage of the program is shown below.

C> calc <switch> <n> <m>

Where, **n** and **m** are two integer operands. **switch** can be any one of the arithmetic or comparison operators. If arithmetic operator is supplied, the output should be the result of the operation. If comparison operator is supplied then the output should be **True** or **False**.

14 Operations On Bits

- Bitwise Operators
  - One's Complement Operator
  - Right Shift Operator
  - Left Shift Operator
  - Bitwise AND Operator
  - Bitwise OR Operator
  - Bitwise XOR Operator
- The *showbits( )* Function
- Summary
- Exercise

So far we have dealt with characters, integers, floats and their variations. The smallest element in memory on which we are able to operate as yet is a byte; and we operated on it by making use of the data type **char**. However, we haven't attempted to look within these data types to see how they are constructed out of individual bits, and how these bits can be manipulated. Being able to operate on a bit level, can be very important in programming, especially when a program must interact directly with the hardware. This is because, the programming languages are byte oriented, whereas hardware tends to be bit oriented. Let us now delve inside the byte and see how it is constructed and how it can be manipulated effectively. So let us take apart the byte... bit by bit.

Bitwise Operators

One of C's powerful features is a set of bit manipulation operators. These permit the programmer to access and manipulate individual bits within a piece of data. The various Bitwise Operators available in C are shown in Figure 14.1.

Operator	Meaning
~	One's complement
>>	Right shift
<<	Left shift
&	Bitwise AND
|	Bitwise OR
^	Bitwise XOR(Exclusive OR)

Figure 14.1

These operators can operate upon **int**s and **char**s but not on **float**s and **double**s. Before moving on to the details of the operators, let

us first take a look at the bit numbering scheme in integers and characters. Bits are numbered from zero onwards, increasing from right to left as shown below:

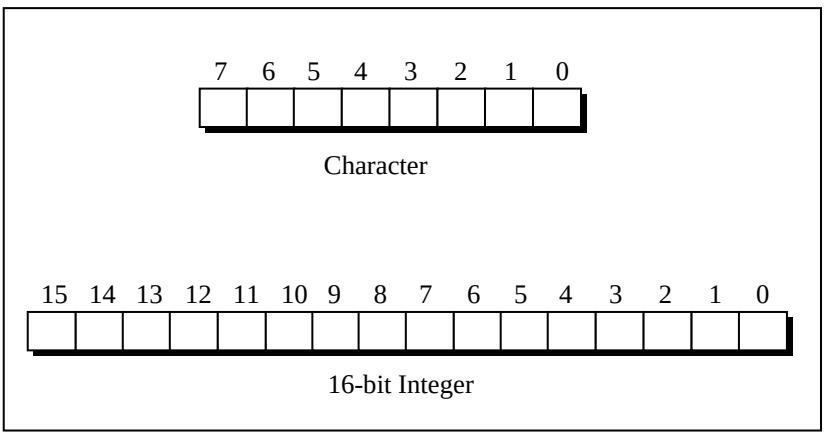

Figure 14.2

Throughout this discussion of bitwise operators we are going to use a function called **showbits( )**, but we are not going to show you the details of the function immediately. The task of **showbits( )** is to display the binary representation of any integer or character value.

We begin with a plain-jane example with **showbits( )** in action.

```
/* Print binary equivalent of integers using showbits( ) function */
main( )
{
    int  j ;

    for ( j = 0 ; j <<= 5 ; j++ )
    {
        printf ( "\nDecimal %d is same as binary ", j ) ;
        showbits ( j ) ;
    }
```

```
}
```

And here is the output...

```
Decimal 0 is same as binary 0000000000000000
Decimal 1 is same as binary 0000000000000001
Decimal 2 is same as binary 0000000000000010
Decimal 3 is same as binary 0000000000000011
Decimal 4 is same as binary 0000000000000100
Decimal 5 is same as binary 0000000000000101
```

Let us now explore the various bitwise operators one by one.

One's Complement Operator

On taking one's complement of a number, all 1's present in the number are changed to 0's and all 0's are changed to 1's. For example one's complement of 1010 is 0101. Similarly, one's complement of 1111 is 0000. Note that here when we talk of a number we are talking of binary equivalent of the number. Thus, one's complement of 65 means one's complement of 0000 0000 0100 0001, which is binary equivalent of 65. One's complement of 65 therefore would be, 1111 1111 1011 1110. One's complement operator is represented by the symbol ~. Following program shows one's complement operator in action.

```
main( )
{
    int   j, k ;

    for ( j = 0 ; j <= 3 ; j++ )
    {
        printf ( "\nDecimal %d is same as binary ", j ) ;
        showbits ( j ) ;

        k = ~j ;
        printf ( "\nOne's complement of %d is ", j ) ;
```

```
            showbits ( k ) ;
        }
}
```

And here is the output of the above program...

```
Decimal 0 is same as binary 0000000000000000
One's complement of 0 is 1111111111111111
Decimal 1 is same as binary 0000000000000001
One's complement of 1 is 1111111111111110
Decimal 2 is same as binary 0000000000000010
One's complement of 2 is 1111111111111101
Decimal 3 is same as binary 0000000000000011
One's complement of 3 is 1111111111111100
```

In real-world situations where could the one's complement operator be useful? Since it changes the original number beyond recognition, one potential place where it can be effectively used is in development of a file encryption utility as shown below:

```
/* File encryption utility */
#include "stdio.h"
main( )
{
    encrypt( ) ;
}

encrypt( )
{
    FILE  *fs, *ft ;
    char   ch ;

    fs = fopen ( "SOURCE.C", "r" ) ;  /* normal file */
    ft = fopen ( "TARGET.C", "w" ) ;  /* encrypted file */

    if ( fs == NULL || ft == NULL )
    {
```

```
        printf ( "\nFile opening error!" ) ;
        exit ( 1 ) ;
    }

    while ( ( ch = getc ( fs ) ) != EOF )
        putc ( ~ch, ft ) ;

    fclose ( fs ) ;
    fclose ( ft ) ;
}
```

How would you write the corresponding decrypt function? Would there be any problem in tackling the end of file marker? It may be recalled here that the end of file in text files is indicated by a character whose ASCII value is 26.

Right Shift Operator

The right shift operator is represented by >>. It needs two operands. It shifts each bit in its left operand to the right. The number of places the bits are shifted depends on the number following the operator (i.e. its right operand).

Thus, **ch >> 3** would shift all bits in **ch** three places to the right. Similarly, **ch >> 5** would shift all bits 5 places to the right.

For example, if the variable **ch** contains the bit pattern 11010111, then, **ch >> 1** would give 01101011 and **ch >> 2** would give 00110101.

Note that as the bits are shifted to the right, blanks are created on the left. These blanks must be filled somehow. They are always filled with zeros. The following program demonstrates the effect of right shift operator.

```
main( )
{
```

```
    int  i = 5225, j, k ;

    printf ( "\nDecimal %d is same as binary ", i ) ;
    showbits ( i ) ;

    for ( j = 0 ; j <= 5 ; j++ )
    {
        k = i >>j ;
        printf ( "\n%d right shift %d gives ", i, j ) ;
        showbits ( k ) ;
    }
}
```

The output of the above program would be...

```
Decimal 5225 is same as binary 0001010001101001
5225 right shift 0 gives 0001010001101001
5225 right shift 1 gives 0000101000110100
5225 right shift 2 gives 0000010100011010
5225 right shift 3 gives 0000001010001101
5225 right shift 4 gives 0000000101000110
5225 right shift 5 gives 0000000010100011
```

Note that if the operand is a multiple of 2 then shifting the operand one bit to right is same as dividing it by 2 and ignoring the remainder. Thus,

```
64 >> 1 gives 32
64 >> 2 gives 16
128 >> 2 gives 32
```

but,

```
27 >> 1 is 13
49 >> 1 is 24 .
```

A Word of Caution

In the explanation **a >> b** if **b** is negative the result is unpredictable. If **a** is negative than its left most bit (sign bit) would be 1. On some computer right shifting **a** would result in extending the sign bit. For example, if **a** contains -1, its binary representation would be 1111111111111111. Without sign extension, the operation **a >> 4** would be 0000111111111111. However, on the machine on which we executed this expression the result turns out to be 1111111111111111. Thus the sign bit 1 continues to get extended.

Left Shift Operator

This is similar to the right shift operator, the only difference being that the bits are shifted to the left, and for each bit shifted, a 0 is added to the right of the number. The following program should clarify my point.

```
main( )
{
    int  i = 5225, j, k ;

    printf ( "\nDecimal %d is same as ", i ) ;
    showbits ( i ) ;

    for ( j = 0 ; j <= 4 ; j++ )
    {
        k = i <<j ;
        printf ( "\n%d left shift %d gives ", i, j ) ;
        showbits ( k ) ;
    }
}
```

The output of the above program would be...

```
Decimal 5225 is same as binary 0001010001101001
```

```
5225 left shift 0 gives 0001010001101001
5225 left shift 1 gives 0010100011010010
5225 left shift 2 gives 0101000110100100
5225 left shift 3 gives 1010001101001000
5225 left shift 4 gives 0100011010010000
```

Having acquainted ourselves with the left shift and right shift operators, let us now find out the practical utility of these operators.

In DOS/Windows the date on which a file is created (or modified) is stored as a 2-byte entry in the 32 byte directory entry of that file. Similarly, a 2-byte entry is made of the time of creation or modification of the file. Remember that DOS/Windows doesn't store the date (day, month, and year) of file creation as a 8 byte string, but as a codified 2 byte entry, thereby saving 6 bytes for each file entry in the directory. The bitwise distribution of year, month and date in the 2-byte entry is shown in Figure 14.3.

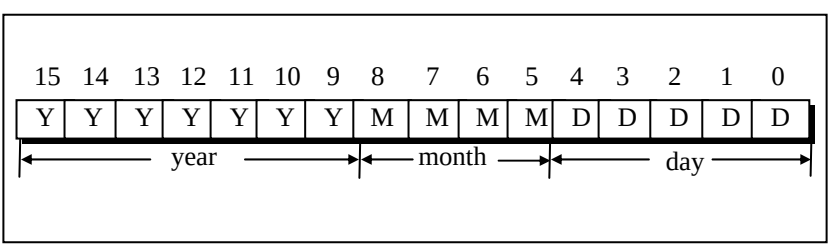

Figure 14.3

DOS/Windows converts the actual date into a 2-byte value using the following formula:

```
date = 512 * ( year - 1980 ) + 32 * month + day
```

Suppose 09/03/1990 is the date, then on conversion the date will be,

```
date = 512 * ( 1990 - 1980 ) + 32 * 3 + 9 = 5225
```

The binary equivalent of 5225 is 0001 0100 0110 1001. This binary value is placed in the date field in the directory entry of the file as shown below.

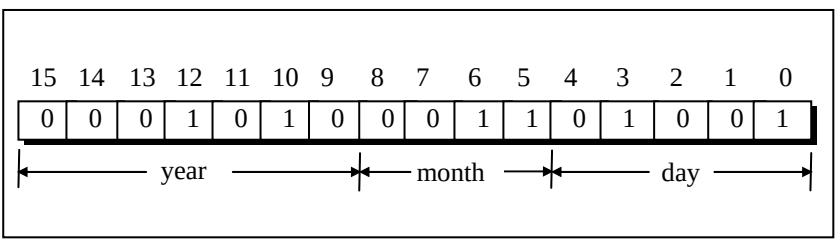

Figure 14.4

Just to verify this bit distribution, let us take the bits representing the month,

```
month = 0011
      = 1 * 2 + 1 * 1
      = 3
```

Similarly, the year and the day can also be verified.

When we issue the command DIR or use Windows Explorer to list the files, the file's date is again presented on the screen in the usual date format of mm/dd/yy. How does this integer to date conversion take place? Obviously, using left shift and right shift operators.

When we take a look at Figure 14.4 depicting the bit pattern of the 2- byte date field, we see that the year, month and day exist as a bunch of bits in contiguous locations. Separating each of them is a matter of applying the bitwise operators.

For example, to get year as a separate entity from the two bytes entry we right shift the entry by 9 to get the year. Just see, how...

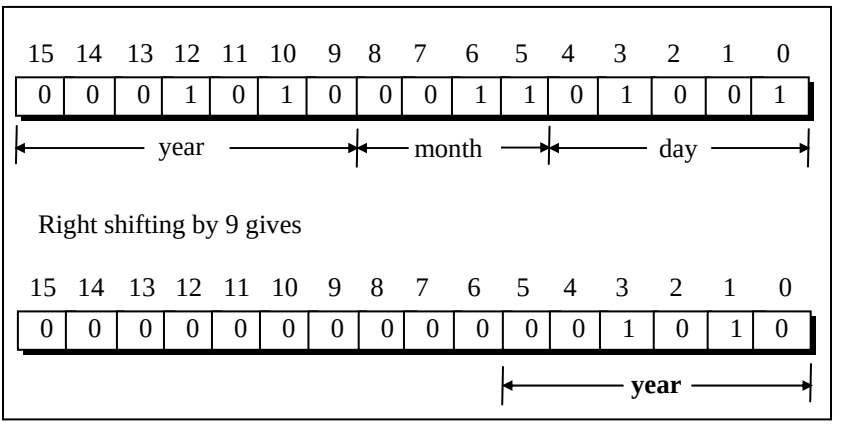

Figure 14.5

On similar lines, left shifting by 7, followed by right shifting by 12 yields month.

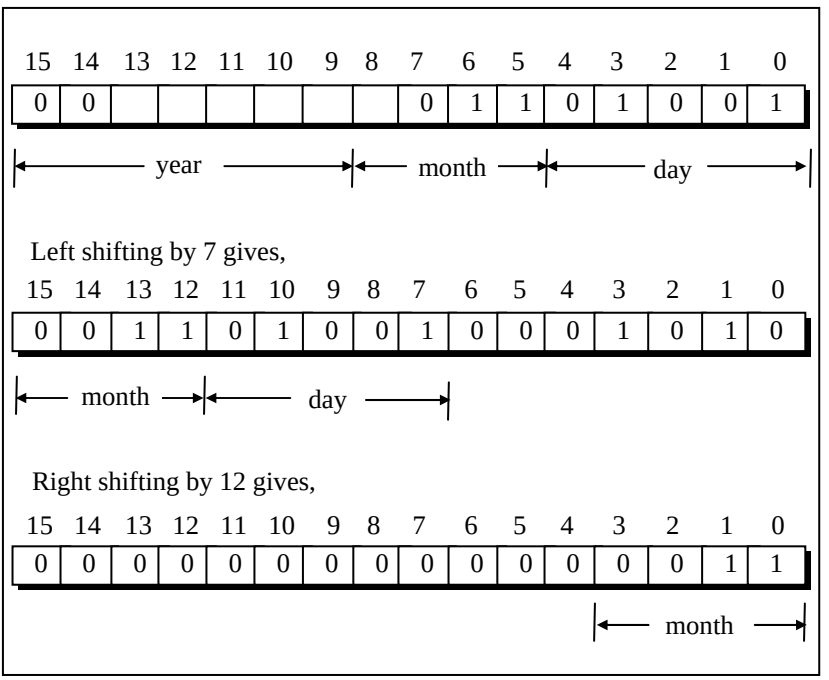

Figure 14.6

Finally, for obtaining the day, left shift date by 11 and then right shift the result by 11. Left shifting by 11 gives 0100100000000000. Right shifting by 11 gives 0000000000001001.

This entire logic can be put into a program as shown below:

```
/* Decoding date field in directory entry using bitwise operators */
main( )
{
    unsigned int  d = 9, m = 3, y = 1990, year, month, day, date ;

    date = ( y - 1980 ) * 512 + m * 32 + d ;
    printf ( "\nDate = %u", date ) ;
```

```
    year = 1980 + ( date >> 9 ) ;
    month = ( (date << 7 ) >> 12 ) ;
    day = ( (date << 11 ) >> 11 ) ;
    printf ( "\nYear = %u ", year ) ;
    printf ( "Month = %u ", month ) ;
    printf ( "Day = %u", day ) ;
}
```

And here is the output...

```
Date = 5225
Year = 1990  Month = 3  Day = 9
```

Bitwise AND Operator

This operator is represented as **&**. Remember it is different than **&&**, the logical AND operator. The **&** operator operates on two operands. While operating upon these two operands they are compared on a bit-by-bit basis. Hence both the operands must be of the same type (either **char** or **int**). The second operand is often called an AND mask. The **&** operator operates on a pair of bits to yield a resultant bit. The rules that decide the value of the resultant bit are shown below:

First bit	Second bit	First bit & Second bit
0	0	0
0	1	0
1	0	0
1	1	1

Figure 14.7

This can be represented in a more understandable form as a 'Truth Table' shown in Figure 14.8.

&	0	1
0	0	0
1	0	1

Figure 14.8

The example given below shows more clearly what happens while ANDing one operand with another. The rules given in the Figure 14.8 are applied to each pair of bits one by one.

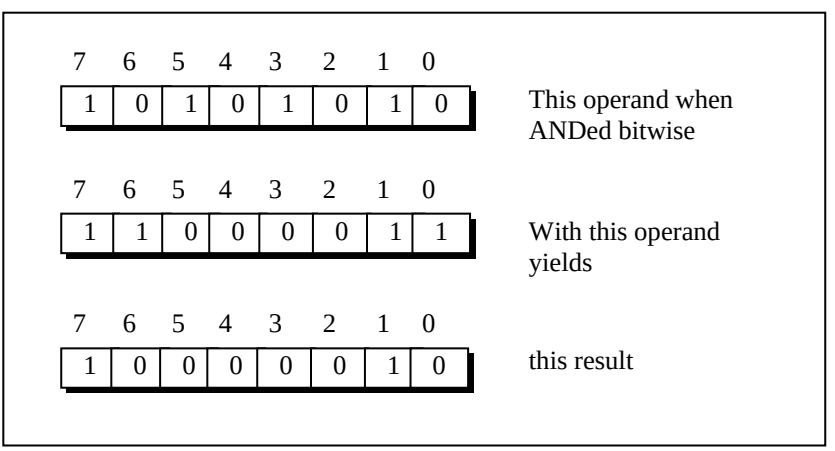

Figure 14.9

Work through the Truth Table and confirm that the result obtained is really correct.

Thus, it must be clear that the operation is being performed on individual bits, and the operation performed on one pair of bits is

completely independent of the operation performed on the other pairs.

Probably, the best use of the AND operator is to check whether a particular bit of an operand is ON or OFF. This is explained in the following example.

Suppose, from the bit pattern 10101101 of an operand, we want to check whether bit number 3 is ON (1) or OFF (0). Since we want to check the bit number 3, the second operand for the AND operation should be $1 * 2^3$, which is equal to 8. This operand can be represented bitwise as 00001000.

Then the ANDing operation would be,

```
10101101        Original bit pattern
00001000        AND mask
--------------
00001000        Resulting bit pattern
```

The resulting value we get in this case is 8, i.e the value of the second operand. The result turned out to be 8 since the third bit of the first operand was ON. Had it been OFF, the bit number 3 in the resulting bit pattern would have evaluated to 0 and the complete bit pattern would have been 00000000.

Thus, depending upon the bit number to be checked in the first operand we decide the second operand, and on ANDing these two operands the result decides whether the bit was ON or OFF. If the bit is ON (1), the resulting value turns out to be a non-zero value which is equal to the value of second operand, and if the bit is OFF (0) the result is zero as seen above. The following program puts this logic into action.

```
/* To test whether a bit in a number is ON or OFF */
main( )
{
```

```
    int  i = 65, j ;

    printf ( "\nvalue of i = %d", i ) ;
    j = i & 32 ;

    if ( j == 0 )
        printf ( "\nand its fifth bit is off" ) ;
    else
        printf ( "\nand its fifth bit is on" ) ;

    j = i & 64 ;

    if ( j == 0 )
        printf ( "\nwhereas its sixth bit is off" ) ;
    else
        printf ( "\nwhereas its sixth bit is on" ) ;
}
```

And here is the output...

```
Value of i = 65
and its fifth bit is off
whereas its sixth bit is on
```

In every file entry present in the directory, there is an attribute byte. The status of a file is governed by the value of individual bits in this attribute byte. The AND operator can be used to check the status of the bits of this attribute byte. The meaning of each bit in the attribute byte is shown in Figure 14.10.

Bit numbers								Meaning
7	6	5	4	3	2	1	0	
.	.	.	.	.	.	.	1	Read only
.	.	.	.	.	.	1	.	Hidden
.	.	.	.	.	1	.	.	System
.	.	.	.	1	.	.	.	Volume label entry
.	.	.	1	.	.	.	.	Sub-directory entry
.	.	1	.	.	.	.	.	Archive bit
.	1	.	.	.	.	.	.	Unused
1	.	.	.	.	.	.	.	Unused

Figure 14.10

Now, suppose we want to check whether a file is a hidden file or not. A hidden file is one, which is never shown in the directory, even though it exists on the disk. From the above bit classification of attribute byte, we only need to check whether bit number 1 is ON or OFF.

So, our first operand in this case becomes the attribute byte of the file in question, whereas the second operand is the $1 * 2^1 = 2$, as discussed earlier. Similarly, it can be checked whether the file is a system file or not, whether the file is read-only file or not, and so on.

The second, and equally important use of the AND operator is in changing the status of the bit, or more precisely to switch OFF a particular bit.

If the first operand happens to be 00000111, then to switch OFF bit number 1, our AND mask bit pattern should be 11111101. On applying this mask, we get,

```
00000111    Original bit pattern
11111101    AND mask
--------------
00000101    Resulting bit pattern
```

Here in the AND mask we keep the value of all other bits as 1 except the one which is to be switched OFF (which is purposefully kept as 0). Therefore, irrespective of whether the first bit is ON or OFF previously, it is switched OFF. At the same time the value 1 provided in all the other bits of the AND mask (second operand) keeps the bit values of the other bits in the first operand unaltered.

Let's summarize the uses of bitwise AND operator:

(a) It is used to check whether a particular bit in a number is ON or OFF.
(b) It is used to turn OFF a particular bit in a number.

Bitwise OR Operator

Another important bitwise operator is the OR operator which is represented as |. The rules that govern the value of the resulting bit obtained after ORing of two bits is shown in the truth table below.

|	0	1
0	0	1
1	1	1

Figure 14.11

Using the Truth table confirm the result obtained on ORing the two operands as shown below.

```
11010000    Original bit pattern
00000111    OR mask
------------
11010111    Resulting bit pattern
```

Bitwise OR operator is usually used to put ON a particular bit in a number.

Let us consider the bit pattern 11000011. If we want to put ON bit number 3, then the OR mask to be used would be 00001000. Note that all the other bits in the mask are set to 0 and only the bit, which we want to set ON in the resulting value is set to 1.

Bitwise XOR Operator

The XOR operator is represented as ^ and is also called an Exclusive OR Operator. The OR operator returns 1, when any one of the two bits or both the bits are 1, whereas XOR returns 1 only if one of the two bits is 1. The truth table for the XOR operator is given below.

^	0	1
0	0	1
1	1	0

Figure 14.12

XOR operator is used to toggle a bit ON or OFF. A number XORed with another number twice gives the original number. This is shown in the following program.

```
main( )
{
      int  b = 50 ;

      b = b ^ 12 ;
      printf ( "\n%d", b ) ;  /* this will print 62 */

      b = b ^ 12 ;
      printf ( "\n%d", b ) ;  /* this will print 50 */
}
```

The *showbits( )* Function

We have used this function quite often in this chapter. Now we have sufficient knowledge of bitwise operators and hence are in a position to understand it. The function is given below followed by a brief explanation.

```
showbits ( int  n )
{
      int  i, k, andmask ;

      for ( i = 15 ; i >= 0 ; i-- )
      {
            andmask = 1 << i ;
            k = n & andmask ;

            k == 0 ? printf ( "0" ) : printf ( "1" ) ;
      }
}
```

All that is being done in this function is using an AND operator and a variable **andmask** we are checking the status of individual bits. If the bit is OFF we print a 0 otherwise we print a 1.

First time through the loop, the variable **andmask** will contain the value 1000000000000000, which is obtained by left-shifting 1,

fifteen places. If the variable **n'**s most significant bit is 0, then **k** would contain a value 0, otherwise it would contain a non-zero value. If **k** contains 0 then **printf( )** will print out 0 otherwise it will print out 1.

On the second go-around of the loop, the value of **i** is decremented and hence the value of **andmask** changes, which will now be 0100000000000000. This checks whether the next most significant bit is 1 or 0, and prints it out accordingly. The same operation is repeated for all bits in the number.

Summary

(a) To help manipulate hardware oriented data—individual bits rather than bytes a set of bitwise operators are used.
(b) The bitwise operators include operators like one's complement, right-shift, left-shift, bitwise AND, OR, and XOR.
(c) The one's complement converts all zeros in its operand to 1s and all 1s to 0s.
(d) The right-shift and left-shift operators are useful in eliminating bits from a number—either from the left or from the right.
(e) The bitwise AND operators is useful in testing whether a bit is on/off and in putting off a particular bit.
(f) The bitwise OR operator is used to turn on a particular bit.
(g) The XOR operator works almost same as the OR operator except one minor variation.

Exercise

[A] Answer the following:

(a) The information about colors is to be stored in bits of a **char** variable called **color**. The bit number 0 to 6, each represent 7 colors of a rainbow, i.e. bit 0 represents violet, 1 represents

indigo, and so on. Write a program that asks the user to enter a number and based on this number it reports which colors in the rainbow does the number represents.

(b) A company planning to launch a new newspaper in market conducts a survey. The various parameters considered in the survey were, the economic status (upper, middle, and lower class) the languages readers prefer (English, Hindi, Regional language) and category of paper (daily, supplement, tabloid). Write a program, which reads data of 10 respondents through keyboard, and stores the information in an array of integers. The bit-wise information to be stored in an integer is given below:

Bit Number	Information
0	Upper class
1	Middle class
2	Lower class
3	English
4	Hindi
5	Regional Language
6	Daily
7	Supplement
8	Tabloid

At the end give the statistical data for number of persons who read English daily, number of upper class people who read tabloid and number of regional language readers.

(c) In an inter-college competition, various sports and games are played between different colleges like cricket, basketball, football, hockey, lawn tennis, table tennis, carom and chess. The information regarding the games won by a particular college is stored in bit numbers 0, 1, 2, 3, 4, 5, 6, 7 and 8 respectively of an integer variable called **game**. The college

that wins in 5 or more than 5 games is awarded the Champion of Champions trophy. If a number is entered through the keyboard, then write a program to find out whether the college won the Champion of the Champions trophy or not, along with the names of the games won by the college.

(d) An animal could be either a canine (dog, wolf, fox, etc.), a feline (cat, lynx, jaguar, etc.), a cetacean (whale, narwhal, etc.) or a marsupial (koala, wombat, etc.). The information whether a particular animal is canine, feline, cetacean, or marsupial is stored in bit number 0, 1, 2 and 3 respectively of a integer variable called **type**. Bit number 4 of the variable **type** stores the information about whether the animal is Carnivore or Herbivore.

For the following animal, complete the program to determine whether the animal is a herbivore or a carnivore. Also determine whether the animal is a canine, feline, cetacean or a marsupial.

```
struct animal
{
    char  name[30] ;
    int  type ;
}
struct animal  a = { "OCELOT", 18 } ;
```

(e) The time field in the directory entry is 2 bytes long. Distribution of different bits which account for hours, minutes and seconds is given below. Write a function which would receive the two-byte time entry and return to the calling function, the hours, minutes and seconds.

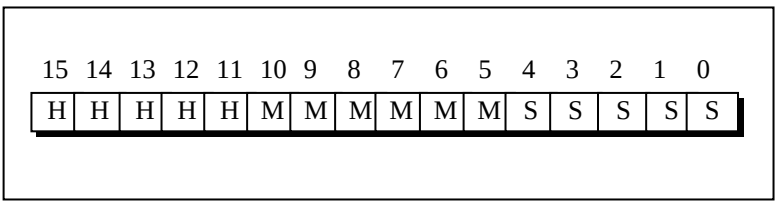

Figure 14.13

(f) In order to save disk space information about student is stored in an integer variable. If bit number 0 is on then it indicates Ist year student, bit number 1 to 3 stores IInd year, IIIrd year and IVth year student respectively. The bit number 4 to 7 stores stream Mechanical, Chemical, Electronics and IT. Rest of the bits store room number. Based on the given data, write a program that asks for the room number and displays the information about the student, if its data exists in the array. The contents of array are,

```
int data[ ] = {  273, 548, 786, 1096 } ;
```

(g) What will be the output of the following program:

```
main( )
{
    int  i = 32, j = 65, k, l, m, n, o, p ;
    k = i | 35 ;   l = ~k ;   m = i & j ;
    n = j ^ 32 ;   o = j << 2 ;   p = i >> 5 ;
    printf ( "\nk = %d l = %d m = %d", k, l, m ) ;
    printf ( "\nn = %d o = %d p = %d", n, o, p ) ;
}
```